

claude-windows / a9fd4426-f234-4e4a-bfb4-1f2c8255c661

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\`  
a9fd4426-f234-4e4a-bfb4-1f2c8255c661\subagents\workflows\wf_076beba8-59c\agent-  
aa32a59bd8ff7052a.jsonl`  
- SHA-256: `b1f5d6172add7ed1fee7cddda4417094aeea0d4c47963da59e75728e552773c1`  
- Source modified: `2026-05-31T04:18:17+00:00`  
- Imported at: `2026-07-05T16:48:20+00:00`  
- project: `wf_076beba8-59c`  
- session_id: `a9fd4426-f234-4e4a-bfb4-1f2c8255c661`
```

Transcript

1. user (2026-05-31T04:17:29.377Z)

Source Extractor

Research question: "Identify the best LOCAL (self-hosted, single NVIDIA CUDA GPU) vision / document-AI / OCR models ? and, as a clearly-flagged approval-gated fallback only, paid cloud API services ? for parsing BANK STATEMENT PDFs into structured transaction tables (date, narration/description, withdrawal/debit, deposit/credit, running balance), for a LOCAL-FIRST personal-finance tool ("muneem").

Target documents: Indian bank statements (HDFC, Axis, ICICI, AU Small Finance) ? dense, multi-column, often SEMI-RULED or borderless tables, sometimes multiple accounts per statement; both born-digital (text layer present) and the harder cases where pdfplumber's extract_tables/coordinate clustering is unreliable (e.g. Axis, whose column layout varies between statements).

HARD CONSTRAINTS (a model that violates these is unusable for us):

1. MUST run 100% LOCALLY for statement contents (highly sensitive). NO cloud APIs by default. Paid/cloud is acceptable ONLY as an explicitly-approved fallback ? still report the best cloud options but label them clearly as "cloud/sensitive ? approval-gated, not default".
2. MUST be usable from Python (transformers / vLLM / Ollama / ONNX Runtime / native lib).
3. License MUST be permissive & commercially safe for BOTH the CODE and the MODEL WEIGHTS: MIT, BSD, Apache-2.0, ISC, HPND. AVOID and explicitly FLAG anything GPL/LGPL/AGPL, CC-BY-NC / non-commercial, research-only, or custom "community"/acceptable-use licenses with restrictions. This is critical ? many document-AI models and their training datasets are non-commercial (e.g. LayoutLMV3 weights are CC-BY-NC; Surya and Marker use GPL-or-commercial dual licensing; several VLM weights ship custom restrictive licenses). Verify the WEIGHTS license, not just the repo license.
4. Hardware: a single consumer/prosumer NVIDIA GPU. Give VRAM tiers (~8-12 GB consumer, ~16-24 GB) and which models/quantizations fit each.

Evaluate and compare these LOCAL approaches ? for each give: code license, MODEL-WEIGHTS license (state explicitly), latest release / maintenance status as of 2025-2026, VRAM need + quantization options, Python integration path, and reliability specifically on DENSE FINANCIAL TABLES / multi-column statements:

- Open vision-language models (VLMs) for document/table understanding: Qwen2-VL and Qwen2.5-VL (note per-size license ? which sizes are Apache-2.0 vs restricted), MiniCPM-V 2.6, InternVL2 / 2.5, Microsoft Phi-3.5-vision, Mistral Pixtral, Meta Llama-3.2-Vision (flag the Llama Community License restrictions), Moondream2, GOT-OCR2.0, olmOCR and other OCR-specialized VLMs.
- OCR-specialist / document-AI pipelines: PaddleOCR + PP-Structure (table recognition), docTR, EasyOCR, Tesseract (already used), Microsoft Table Transformer / TATR (check code AND PubTables-1M dataset/weights license), Donut, Surya (FLAG license), Marker (FLAG license), MinerU / PDF-Extract-Kit, RapidOCR.
- Table-structure recognition specifically (cell/grid reconstruction for borderless tables): TATR, PP-Structure, UniTable, etc.

Also assess: for SEMI-RULED / borderless bank tables, is a prompt-driven VLM (page image -> JSON transactions) more reliable than a classic OCR + table-structure pipeline? What are the accuracy/hallucination risks of VLMs on financial NUMBERS (transposed digits, dropped/duplicated rows), and how should output be VERIFIED? (muneem already machine-checks parses by balance reconciliation ? opening + sum(deposits) - sum(withdrawals) == closing, plus per-row running-balance walks and cross-checks against the bank's printed summary ? so any model's output can be automatically validated, which should inform the recommendation.)

Finally, briefly cover the best PAID/CLOUD fallbacks (clearly flagged approval-gated for sensitive data): AWS Textract (AnalyzeExpense / Tables), Google Document AI, Azure AI Document Intelligence, Reducto, LlamaParse, Anthropic/OpenAI vision ? note any self-hosting option and data-handling terms.

Deliver a RANKED shortlist of the most promising LOCAL options for muneem (NVIDIA GPU, Python, commercial-license-safe), each with license (code + weights), VRAM tier, maturity, strengths/weaknesses for dense bank-statement tables, and a concrete recommendation: which local model(s) to prototype for muneem's statement parsing and how they'd combine with the existing deterministic parsers + balance-reconciliation verification (e.g. VLM as a fallback/cross-check when deterministic parsing fails to reconcile). Cite sources and call out any license uncertainty explicitly."

Fetch and extract key claims from this source:

```
**URL:** https://nanonets.com/blog/vision-language-model-vlm-for-data-extraction/  
**Title:** Best Vision Language Models for Document Data Extraction (Nanonets)  
**Found via:** Open VLMs for document/table extraction (broad/primary) search
```

Task

1. Use WebFetch to retrieve the page content.
2. Assess source quality: primary research/institution? secondary reporting? blog/opinion? forum? unreliable?
3. Extract 2-5 FALSIFIABLE claims that bear on the research question. Each claim must:
 - be a concrete, checkable statement (not vague generalities)
 - include a direct quote from the source as support
 - be rated central/supporting/tangential to the research question
4. Note publish date if available.

If the fetch fails or the page is irrelevant/paywalled, return claims: [] and sourceQuality: "unreliable".

Structured output only.

2. assistant (2026-05-31T04:17:36.800Z)

I'll fetch the page content first.

3. assistant (2026-05-31T04:17:39.875Z)

I have the WebFetch tool. Now let me fetch the page.

4. user (2026-05-31T04:17:52.520Z)

Vision-Language Models for Document Data Extraction - Claims Summary

Article Date: September 19, 2024

Source: Nanonets Blog (vendor blog for document processing/OCR services)

VLMS Named and Evaluated

Open Source Models:

- Qwen2-VL-2B
- MiniCPM-V-2.6
- Bunny-v1_1-Llama-3-8B-V

****Closed Source Models:****

- GPT-4o-Mini
- Claude 3.5 Sonnet
- Gemini 1.5 Flash

(Florence2 was tested but excluded due to poor results)

Key Claims About Performance

****On Accuracy:****

- Regarding Qwen2: "The overall accuracies were the best among the three open VLMs"
- About MiniCPM: "Results were reliable and consistent across both the datasets"
- On Bunny: "Predictions can vary significantly, from highly accurate to very poor"

****On Hallucination:****

- Gemini described as having "the least hallucinations among the VLMs"
- Bunny's low precision indicated "it's limited reasoning capacity, leading to it returning random hallucinated data"
- Open-source models noted as "susceptible to hallucination" on complex tasks

****On Table Extraction:****

- "closed source models edging out on open source variants in the precision score"
- Bunny's "high recall suggests...able to read the OCR content properly but the low precision is indicative of...limited reasoning capacity"

Model Specifications

- ****Qwen2-VL-2B:**** 6GB VRAM, 8.73s prediction time
- ****MiniCPM-V-2.6:**** 20GB VRAM, 5s prediction time
- ****Bunny:**** 18GB VRAM, 3.37s prediction time

Key Recommendations

"Qwen is the best option for open source models while Gemini's free tier is the most cost effective option for short term"

****Prompting importance:**** "Prompts need to be carefully evaluated for maximum efficiency and minimal hallucination"

****Local vs. Cloud:**** The article advocates for local deployment of open-source models to control costs, noting that GPU compute pricing should be compared against API costs.

5. assistant (2026-05-31T04:17:57.144Z)

I have the content. Let me assess and extract the key claims.

This is a vendor blog (Nanonets, a commercial document-processing/OCR company), dated September 19, 2024. It contains an empirical comparison of VLMs, which gives it some primary-evaluation value, but it is a vendor blog with commercial interest, so I'll rate it "blog". The claims are concrete and falsifiable (specific VRAM numbers, accuracy/hallucination findings, recommendations).

6. user (2026-05-31T04:18:04.987Z)

Structured output provided successfully

7. assistant (2026-05-31T04:18:17.462Z)

I've fetched and analyzed the Nanonets blog source. Summary of what I returned:

****Source quality: blog**** ? This is a vendor blog from Nanonets (a commercial document-processing/OCR company), published September 19, 2024. It does contain a hands-on empirical benchmark, which adds some value, but it's a vendor with commercial interest and the model

coverage is dated (pre-Qwen2.5-VL era).

****Relevance to muneem's research question:**** Moderate. It directly addresses several of the named candidate models and the VLM-vs-numbers hallucination concern, but it does NOT discuss model-weights licensing at all ? a critical gap given muneem's hard licensing constraints. The benchmark tests only Qwen2-VL-2B (not larger sizes), MiniCPM-V-2.6, and Bunny, and uses closed cloud models (GPT-4o-Mini, Claude 3.5 Sonnet, Gemini 1.5 Flash) as comparison.

****Key extracted claims (5):****

1. ****Central**** ? Qwen2-VL-2B had the best overall accuracy among the three open VLMs; Nanonets recommends Qwen as the best open-source option.
2. ****Central**** ? Open VLMs are hallucination-prone on complex tasks; Bunny showed high recall / low precision (reads OCR but weak reasoning ? hallucinated data). Directly relevant to muneem's concern about VLM reliability on financial numbers.
3. ****Supporting**** ? VRAM/latency figures: Qwen2-VL-2B ~6GB, MiniCPM-V-2.6 ~20GB, Bunny ~18GB. Useful for the VRAM-tier analysis, though only the 2B Qwen size is covered.
4. ****Supporting**** ? Closed cloud models edged out open ones on precision for tables; Gemini had the fewest hallucinations.
5. ****Supporting**** ? Prompt tuning materially affects accuracy and hallucination rate.

****Caveats for the synthesizer:**** No license information is provided (the source cannot speak to muneem's MIT/BSD/Apache weights constraint). Only the smallest Qwen size was benchmarked, so it says nothing about Qwen2.5-VL or the larger 7B/72B sizes relevant to the 16-24GB tier. Findings predate olmOCR, GOT-OCR2.0 maturity, and Qwen2.5-VL. The benchmark methodology details (dataset specifics, whether tables were dense/financial) are thin.